

LAB 1 ASSIGNMENT

Jesús Sanz Alonso, Agustín Prieto Páez

November 5, 2024

Contents

1	Introduction	1
2	Overview	1
3	Pseudocode	1
4	Statistical Analysis	3
5	Algorithmic complexity	3
5.0.1	Optimization	4
6	Complexity analysis	4
7	References	5

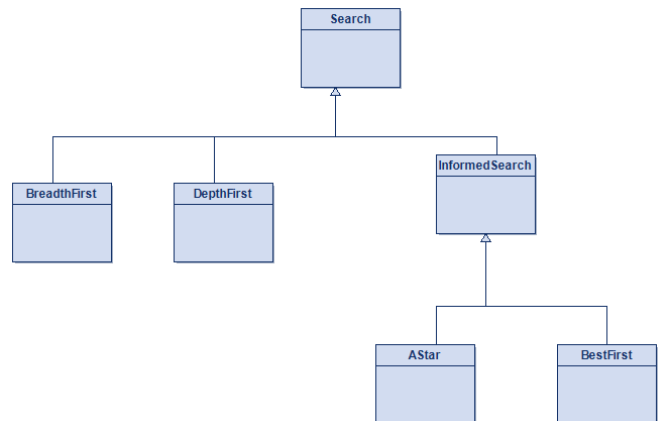
1 Introduction

The term Heuristics comes from the greek expression Heuriskien [1], which means to discover. The same with the expression "Eureka!" that Archimedes claimed when he came out of the river and set his famous Fluids Principle that was named after him. Informed Strategies are more efficient since they can weight different decisions based on a function $f(n)$. This function is going to vary depending on whether we are on A* ($f(n) = g(n) + h(n)$) where $g(n)$ is the accumulated cost on our current node and $h(n)$ is the calculated **heuristics** or we are working with a greedy approach (our function $f(n)$ is going to be determined just by $h(n)$ ($f(n) = h(n)$)). On Uninformed Strategies, we could say $f(n) = 0$.

2 Overview

It is nice to comment that we have used the concept of inheritance to make the code more readable and also not to repeat redundant code. This not only helps for scalability but also for read-

ability. See the inheritance overview in Figure 1.
Figure 1: Class Diagram showing Inheritance.



3 Pseudocode

The two most important functions or methods in this assignment are called **expand** and **search**. Both are in the **Problem** class.

Algorithm 1 Main Algorithm

```

1: function MAIN
2:   Create DataBase
3:   for each directory in currentDirectory do
4:     for each problem in directory do
5:       read problem
6:       declare problem
7:       declare searchStrategy
8:       SEARCH(problem)
9:       write results on the DataBase
10:    end for
11:  end for
12: end function

```

Algorithm 2 Search Algorithm

```

1: function SEARCH ▷ O(n)
2:   open ← CREATENODE(Initial – State)
3:   explored ← ∅
4:   while open ≠ ∅ do
5:     node ← EXTRACT(open)
6:     if node.STATE ∉ explored then
7:       if TESTGOAL(node.STATE) then
8:         return RECOVERPATH(node)
9:       end if
10:    successors ← EXPAND(node) ▷ O(n)
11:    for each successor in successors do ▷ O(n)
12:      INSERT(successor, open)
13:    end for
14:    INSERT(node.STATE, explored)
15:  end if
16: end while
17: return failure
18: end function

```

Algorithm 3 Expand Algorithm

```

1: function EXPAND(node) ▷ O(n)
2:   successors ← ∅
3:   currentIntersection ← dictionary.GET('intersections').GET(node.STATE)
4:   listOrdered ← SORTED(currentIntersection.destinations) ▷ O(n*log(n))
5:   for destination ∈ listOrdered do ▷ O(n)
6:     if destination.GET('id') ∈ explored then
7:       continue
8:     end if
9:     newAction ← ACTION(node.STATE, destination.GET('id'), destination.GET('cost'))
10:    newState ← STATE(node.DESTINATION, destination.GET('longitude'), destination.GET('latitude'))
11:    newNode ← NODE(node, newState, newAction, node.depth + 1, node.ACCUMULATED.COST +
12:      newAction.COST)
13:    NODES_GENERATED += 1
14:    MOMENTO ← NODES_GENERATED
15:    successors.APPEND(newNode)
16:  end for
17: return successors
18: end function

```

4 Statistical Analysis

We have got several parameters we can use to our benefit to compare different strategies based on different metrics. Take Figure 2 below, for instance. We can see how **DepthFirst** search is the strategy with most generated nodes considering all problems of different dimensions. We can see, specifically, in Figure 6 how **DepthFirst** is really bad at huge problems finding paths with really high costs compared to all other search strategies.

Figure 2: Graphic showing the nodes generated of each strategy.



In Figure 3 we can see the execution time on different strategies. Even though there is not much difference between each one, we can say that **DepthFirst** is the more rapid one while **BestFirst** seems to be the one that takes longer. Maybe that is the price we pay for having strategies that are informed, they cost more in terms of Time Complexity. Figure 3: Graphic showing the different execution times of different strategies.

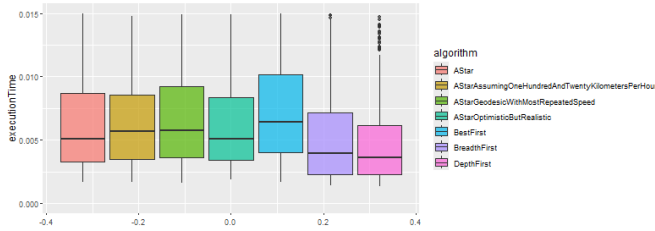


Figure 4: Average Nodes Generated on different search strategies across problems with different dimensionality.

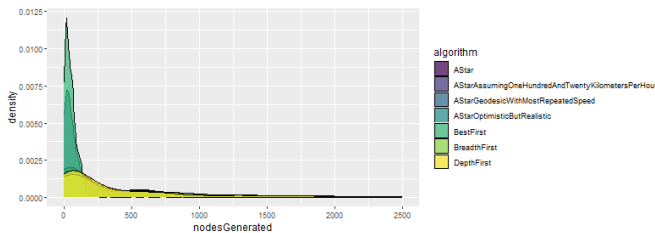


Figure 5: Graphic showing the cost of each strategy.

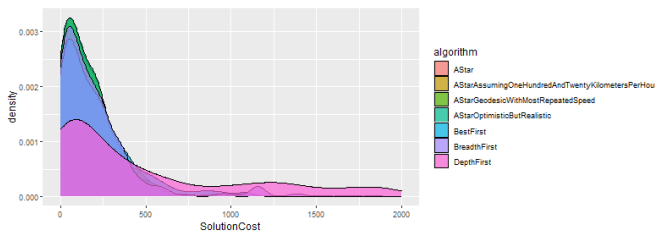


Figure 6: Density graphic showing solution cost of each strategy within a certain type of problem.

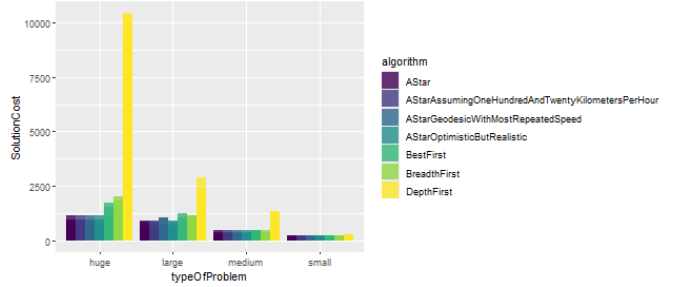
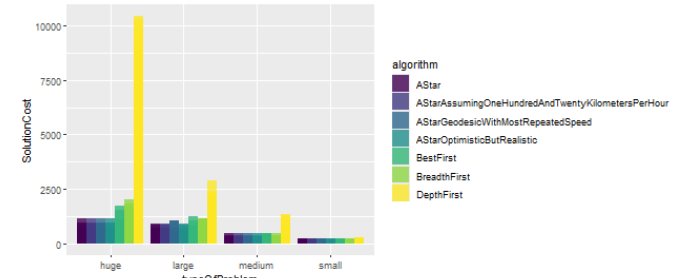
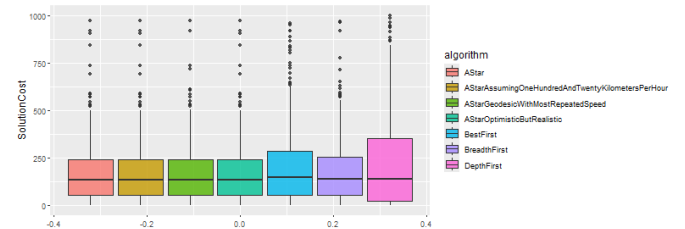


Figure 7: Negative execution time for A* strategy on huge dimension of problems.



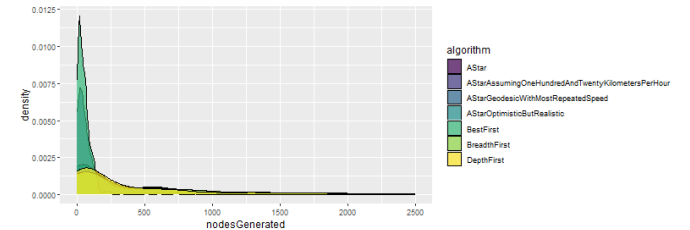
In figure 8 there is not much difference on the cost of the solution on each strategy. **DepthFirst**

Figure 8: Boxplot on different solution costs by each search strategy.



One more time, we can appreciate how **BestFirst** search strategy generates less nodes as any other search strategy.

Figure 9: Density graph on the nodes generated by each strategy.



5 Algorithmic complexity

identifying the key elements we have been working with in class:

- **Solution Data Type:** The solution is a mapping of goods to trucks, where each truck's weight limit is not exceeded.
- **Exhaustivity:** The algorithm explores all possible combinations of assigning goods to trucks until a feasible solution is found or all combinations are exhausted.
- **Dead Node Condition (Backtracking):** If placing a good in a truck would exceed its capacity, the algorithm backtracks to the previous state and explores other possibilities.
- **Live Node Condition:** If the current assignment of goods to trucks does not exceed any truck's capacity and there are goods remaining, the algorithm continues exploring further assignments.
- **Solution Node Condition:** If all goods are successfully assigned to trucks without exceeding any capacity, a solution is found.

5.0.1 Optimization

The following optimizations have been applied to improve the efficiency of the backtracking algorithm:

1. **Feasibility Check:** Before starting the backtracking process, a feasibility check is performed to determine if it's even possible to distribute all goods among the given number of lorries without exceeding the capacity of each lorry or the budget constraint.
2. **Sorting:** The weights of goods are sorted in non-increasing order using the `quickSort` function. Sorting the weights allows the algorithm to try heavier goods first, potentially reducing the search space.
3. **Pruning:** During the backtracking process, if a particular combination of goods and lorries exceeds the load capacity of any lorry, that branch of the search tree is pruned. Similarly, if the cost exceeds the budget constraint, the branch is also pruned. This avoids exploring solutions that are guaranteed to be invalid.
4. **Array to Track Goods Assignment:** An array `selected[NUM_LORRIES][n]` is used to keep track of which goods are assigned to which lorries. This allows the algorithm to print out the weights assigned to each lorry once a valid solution is found.
5. **Avoiding Redundant Permutations:** When checking if a lorry has a load capacity of 0, it breaks the loop to avoid redundant permutations. This helps in reducing unnecessary computations.

6 Complexity analysis

- Analysis of the **computational complexity** of the algorithm in terms of the number of operations executed by the algorithm as a function of the input size n .

Answer: The backtracking algorithm's complexity is analyzed based on the number of operations it executes, which is a function of the input size n , the maximum load capacity of the lorries `MAX_CAPACITY`, and the maximum number of lorries `NUM_LORRIES`. The algorithm consists of several key components, each contributing to the overall complexity as follows:

1. **Sorting the Weights:** The algorithm begins by sorting the weights in non-increasing order. This sorting operation typically takes $O(n \log n)$ time using efficient sorting algorithms like quicksort or mergesort.
2. **Feasibility Check:** Before initiating the backtracking process, a feasibility check is performed to determine if it's possible to distribute all goods among the given number of lorries without exceeding their capacities. This check involves iterating through the weights and ensuring that the sum of weights does not exceed `NUM_LORRIES` times `MAX_CAPACITY`. This operation has a complexity of $O(n)$.
3. **Backtracking Process:** The core of the algorithm is the backtracking process, where it recursively explores all possible combinations of assigning goods to lorries. At each step, it tries to place the current good in each lorry, branching into multiple recursive calls. The maximum depth of the recursion tree is bounded by the number of goods n , and at each level, there are $O(\text{NUM_LORRIES})$ choices. Therefore, the time complexity of the backtracking process is $O(\text{NUM_LORRIES}^n)$.
4. **Solution Output:** After finding a feasible distribution, the algorithm outputs the assigned goods for each lorry along with the total cost. This operation involves iterating through the selected goods and has a complexity of $O(n)$.

Considering these components, the overall complexity of the backtracking algorithm is influenced primarily by the backtracking process, which has a complexity of $O(\text{NUM_LORRIES}^n)$. Additionally, the sorting operation contributes $O(n \log n)$, and the feasibility check and solution output operations contribute $O(n)$ each.

Thus, the total complexity of the algorithm can be described as:

$$T(n, \text{NUM_LORRIES} = 3) \in O(n + n \log n + 3^n) \approx O(3^n)$$

where n represents the number of goods, $MAX_CAPACITY$ is the maximum load capacity of the lorries, and $NUM_LORRIES$ is the maximum number of lorries.

- Description of Quicksort Time Complexity on Average Case

$$T(n) = O(n) + 2T\left(\frac{n}{2}\right)$$

$$n = 2^i$$

$$2^i - 2 \cdot 2^{(i-1)} = 2^i$$

$$r^i - 2 \cdot r^{(i-1)} = 2^i$$

$$r - 2 = 0$$

From left hand side we have got that $r = 2$ and $\alpha = 1$.

From the right hand side we have got that $r = 2$ and $\alpha = 1$.

In brief, $r = 2$ and $\alpha = 2$.

So that we can construct our expression with degree $\alpha - 1$.

$$T(2^i) = 2^i \cdot A + B \cdot 2^i \cdot i$$

On variable change,

$$T(n) = (A + B \cdot \log_2 n) \cdot n$$

Where $\Omega(n)$ and $O(n \log n)$

- Test of the algorithm with increasing values of n and analysis of the time taken by the algorithm (*CPU time*).

Bear in mind that the specific time taken by the algorithm may vary depending on the machine used to run the code and the *GCC compiler* optimization settings. We must disable the optimization

flag to get the real time taken by the algorithm, otherwise the compiler will optimize the code and, therefore, the graph of *CPU time* will not be a representative approximation.

- GPU: *AMD Ryzen 5 3500U with Radeon Vega Mobile Gfx*
- GCC compiler: *gcc (Ubuntu 11.4.0-1ubuntu1 22.04) 11.4.0*
- GCC optimization flag: *#pragma GCC optimize("O0")* (GCC compiler optimization disabled)

```

1 #include <time.h>
2 #pragma GCC optimize("O0")
3
4 if (!feasibilityCheck(weights, n,
5     LOAD_CAPACITY, NUM_LORRIES, BUDGET,
6     COST_PER_KG)) // O(n)
7 {
8     printf("\nIt's not possible to
9     distribute all goods in 3 lorries
10    without exceeding the capacity of 20 Tm
11    each.\n");
12 }
13 else
14 {
15     qsort(weights, n, sizeof(int), compare)
16     ; // O(
17     nlogn)
18     bool result = canPlaceGoods(weights,
19     lorries, 0, n, LOAD_CAPACITY); // O
20     (3~n)
21 }
22
23 end = clock();
24 cpu_time_used = ((double)(end - start)) /
25     CLOCKS_PER_SEC * 1000.0;

```

Listing 1: Optimization settings and time measurement.

- **Conclusion:**

7 References

References

- [1] Roque Luis Palma Méndez José T.; Marín Morales. *Inteligencia artificial : técnicas, métodos y aplicaciones*. McGraw Hill, 2008.